



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Iana Kim Afilleje
Section:	AN42
Professor:	Ms. Crisola Tan
Date:	9/30/2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
(Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
 - Classes and objects
 - If-Else statements

- Loops (for loop)
 - Functions / methods
2. Understand the environment:
 - Two rooms: A and B
 - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output

```
C:\Users\hazel\AppData\Local... x + v
Is room A Dirty or Clean? dirty
Is room B Dirty or Clean? clean

How many steps should the agent run? 5

Step 1: Agent is in room A
Room A has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 2: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 3: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 4: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 5: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}
Press any key to continue . . . |
```

Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
 - Rule 1: If the current room is dirty, clean it.
 - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.

3. After each step, print:
 - The agent's current location
 - The action taken (cleaned/moved)
 - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)

```

C:\Users\hazel\AppData\Local\Programs\Python\Python39-64\python.exe
Is room B Dirty or Clean? dirty
Is room C Dirty or Clean? dirty

How many steps should the agent run? 5

Step 1:
[A-Agent] [B-Dirty] [C-Dirty]
Moving to room B
[A-Clean] [B-Agent] [C-Dirty]

Step 2:
[A-Clean] [B-Agent] [C-Dirty]
Room B has been cleaned.
[A-Clean] [B-Agent] [C-Dirty]

Step 3:
[A-Clean] [B-Agent] [C-Dirty]
Moving to room C
[A-Clean] [B-Clean] [C-Agent]

Step 4:
[A-Clean] [B-Clean] [C-Agent]
Room C has been cleaned.
[A-Clean] [B-Clean] [C-Agent]

Step 5:
[A-Clean] [B-Clean] [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean] [B-Clean] [C-Agent]
Press any key to continue . . .

```

Your Code - Laboratory Tasks

```

import random

# =====
# Task 1: Environment Setup
# =====
class Environment:
    def __init__(self, initial_states=None):
        if initial_states is None:
            # Default state: Rooms A, B, and optional room C

```

```

        self.rooms = {"A": "Dirty", "B": "Dirty", "C": "Dirty"}
    else:
        self.rooms = initial_states

def is_dirty(self, room):
    return self.rooms.get(room) == "Dirty"

def clean_room(self, room):
    self.rooms[room] = "Clean"

def display_state(self):
    return self.rooms

def render_grid(self, agent_location):
    # Text-based grid visualization for Bonus Task
    grid_str = "["
    for room, state in self.rooms.items():
        status = "D" if state == "Dirty" else "C"
        if room == agent_location:
            grid_str += f" >{room}:{status}< "
        else:
            grid_str += f" {room}:{status} "
    grid_str += "]"
    return grid_str

# =====
# Task 2: Rule-Based Vacuum Agent
# =====
class VacuumAgent:
    def __init__(self, initial_location="A"):
        self.location = initial_location

    def perceive_and_act(self, env):
        current_room = self.location

        # Rule 1: If current room is dirty, clean it
        if env.is_dirty(current_room):
            env.clean_room(current_room)
            action = f"Cleaned Room {current_room}"
            # Rule 2: If current room is clean, move to another dirty room
            or next room
        else:
            dirty_rooms = [r for r, state in env.rooms.items() if state
== "Dirty" and r != current_room]
            if dirty_rooms:
                next_room = random.choice(dirty_rooms) # Bonus Task
                logic
            else:

```

```

        # Sequential fallback movement
        all_rooms = list(env.rooms.keys())
        current_idx = all_rooms.index(current_room)
        next_room = all_rooms[(current_idx + 1) %
len(all_rooms)]

        action = f"Moved from Room {self.location} to Room
{next_room}"
        self.location = next_room

    return action

# =====
# Task 3 & Bonus Task: Run Simulation
# =====
def main():
    print("=== Rule-Based Vacuum Cleaner Agent Simulation ===")

    # Task 1: Allow user to input initial states for rooms
    state_a = input("Enter state for Room A (Dirty/Clean) [default:
Dirty]: ").strip().capitalize() or "Dirty"
    state_b = input("Enter state for Room B (Dirty/Clean) [default:
Dirty]: ").strip().capitalize() or "Dirty"

    include_c = input("Include Room C for Bonus Task? (y/n) [default:
y]: ").strip().lower() != 'n'

    initial_states = {"A": state_a, "B": state_b}
    if include_c:
        state_c = input("Enter state for Room C (Dirty/Clean) [default:
Dirty]: ").strip().capitalize() or "Dirty"
        initial_states["C"] = state_c

    env = Environment(initial_states)
    agent = VacuumAgent(initial_location="A")

    try:
        steps = int(input("\nEnter number of steps to run: "))
    except ValueError:
        steps = 5

    print("\n--- Simulation Started ---")
    print(f"Initial State: {env.display_state()}")
    print(f"Grid Layout: {env.render_grid(agent.location)}\n")

    for step in range(1, steps + 1):
        action = agent.perceive_and_act(env)
        print(f"Step {step}:")

```

```

print(f" Agent Location: Room {agent.location}")
print(f" Action Taken   : {action}")
print(f" Environment    : {env.display_state()}")
print(f" Grid View       : {env.render_grid(agent.location)}")
print("-" * 45)

if __name__ == "__main__":
    main()

```

SCREENSHOTS:

```

File Edit Selection View Go Run Terminal Help
Technical Assessment 2
TA2_Aflleje.py
Welcome TA2_Aflleje.py X
TA2_Aflleje.py > main
def main():

Problems Output Debug Console Terminal Ports
PS C:\Users\anie\Aflleje-IanaKim-CS0065-AN42-main\Technical Assessment 2> python -u "c:\Users\anie\Aflleje-IanaKim-CS0065-AN42-main\Techni
1 Assessment 2\TA2_Aflleje.py"
=== Rule-Based Vacuum Cleaner Agent Simulation ===
Enter state for Room A (Dirty/Clean) [default: Dirty]: clean
Enter state for Room B (Dirty/Clean) [default: Dirty]: dirty
Include Room C for Bonus Task? (y/n) [default: y]: y
Enter state for Room C (Dirty/Clean) [default: Dirty]: dirty

Enter number of steps to run: 10

--- Simulation Started ---
Initial State: {'A': 'Clean', 'B': 'Dirty', 'C': 'Dirty'}
Grid Layout: [ >A:C<  B:D  C:D ]

Step 1:
Agent Location: Room C
Action Taken : Moved from Room A to Room C
Environment  : {'A': 'Clean', 'B': 'Dirty', 'C': 'Dirty'}
Grid View    : [ A:C  B:D  >C:D< ]

Step 2:
Agent Location: Room C
Action Taken : Cleaned Room C
Environment  : {'A': 'Clean', 'B': 'Dirty', 'C': 'Clean'}
Grid View    : [ A:C  B:D  >C:C< ]

Step 3:
Agent Location: Room B
Action Taken : Moved from Room C to Room B
Environment  : {'A': 'Clean', 'B': 'Dirty', 'C': 'Clean'}
Grid View    : [ A:C  >B:D<  C:C ]

Step 4:
Agent Location: Room B
Action Taken : Cleaned Room B
Environment  : {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}
Grid View    : [ A:C  >B:C<  C:C ]

```

```
File Edit Selection View Go Run Terminal Help Technical Assessment 2
TA2_Affleje.py
def main():
    1 Assessment 2\TA2_Affleje.py
    Action Taken : Cleaned Room B
    Environment : {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}
    Grid View : [ A:C >B:C< C:C ]

    Step 5:
    Agent Location: Room C
    Action Taken : Moved from Room B to Room C
    Environment : {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}
    Grid View : [ A:C >B:C< C:C ]

    Step 6:
    Agent Location: Room A
    Action Taken : Moved from Room C to Room A
    Environment : {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}
    Grid View : [ >A:C< B:C C:C ]

    Step 7:
    Agent Location: Room B
    Action Taken : Moved from Room A to Room B
    Environment : {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}
    Grid View : [ A:C >B:C< C:C ]

    Step 8:
    Agent Location: Room C
    Action Taken : Moved from Room B to Room C
    Environment : {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}
    Grid View : [ A:C >B:C< C:C ]

    Step 9:
    Agent Location: Room A
    Action Taken : Moved from Room C to Room A
    Environment : {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}
    Grid View : [ >A:C< B:C C:C ]

    Step 10:
    Agent Location: Room B

PS C:\Users\anie\Afilleje-IanaKim-CS0065-AN42-main\Technical Assessment 2> python -u "c:\Users\anie\Afilleje-IanaKim-CS0065-AN42-main\Techni
```

```
File Edit Selection View Go Run Terminal Help Technical Assessment 2
TA2_Affleje.py
def main():
    1 Assessment 2\TA2_Affleje.py
    Action Taken : Moved from Room A to Room B
    Environment : {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}
    Grid View : [ A:C >B:C< C:C ]

    Step 8:
    Agent Location: Room C
    Action Taken : Moved from Room B to Room C
    Environment : {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}
    Grid View : [ A:C >B:C< C:C ]

    Step 9:
    Agent Location: Room A
    Action Taken : Moved from Room C to Room A
    Environment : {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}
    Grid View : [ >A:C< B:C C:C ]

    Step 10:
    Agent Location: Room B
    Action Taken : Moved from Room A to Room B
    Environment : {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}
    Grid View : [ A:C >B:C< C:C ]

PS C:\Users\anie\Afilleje-IanaKim-CS0065-AN42-main\Technical Assessment 2>
```


Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ↓

1. The agent works based on condition action rules set by its environment sensors. Rule one says that if the agent notices that the room it is in is dirty it does the action in that place. Rule two says that if the agent notices that the room it is, in is clean it does the move action to go to a room.
2. The agent checks its surroundings at every time step. It uses basic IF ELSE logic to make decisions. It looks at the room's status tag, whether it is dirty or clean. If the tag says dirty the agent starts the cleaning function. If the tag says clean the agent moves to the room. This process repeats at each step. The agent does not think ahead. It only reacts to what it sees now.
3. If all rooms start in a state rule one never activates. The agent runs rule two right at the time step. After that it keeps moving forth between the clean rooms, for all the remaining steps. It does not perform any cleaning actions all during the entire simulation.
4. To make the model work for rooms the environment dictionary can have more room keys like C, D and E. The movement function can be changed to go through the room keys one, after another or look for any remaining rooms that need cleaning and send the agent to those places using a way to choose the best path.